

# Web and Data Engineering

---

## Lecture 04: HTTP und RESTful APIs

Prof. Dr. Michael Granitzer

# Ziel dieser Vorlesung

HTTP ist das Protokoll des Webs. REST nutzt dieses Protokoll konsequent als Grundlage für API-Design.

Diese Vorlesung verbindet beides: vom **Protokollverständnis über Caching und Sicherheit** bis zum **Entwurf robuster, evolvierbarer APIs**.

## Leitfragen

- Warum ist HTTP mehr als Datentransport?
- Wie strukturieren Header HTTP-Nachrichten und ihre Repräsentationen?
- Wie steuern Header Caching, Sessions und Sicherheit?
- Was unterscheidet REST von beliebigem JSON-über-HTTP?
- Wie modelliert man Ressourcen, URIs und HTTP-Semantik für APIs?
- Wie entwickelt man APIs weiter, ohne Clients zu brechen?

# HTTP Grundlagen

---

**Leitfrage:** Wie können Browser und Server mit einem einfachen, textbasierten Protokoll so kommunizieren, dass das Web weltweit skalierbar bleibt?

# Was sehen Sie hier?

Ein Nutzer ruft die Produktseite auf. Das Netzwerk-Tab zeigt diesen Austausch:

## ↑ REQUEST

```
GET /api/products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
Cookie: session=abc123
```

## ↓ RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300
ETag: "f7e3a1b2"

{"id":42,"name":"Funkkopfhörer","price":79.99}
```

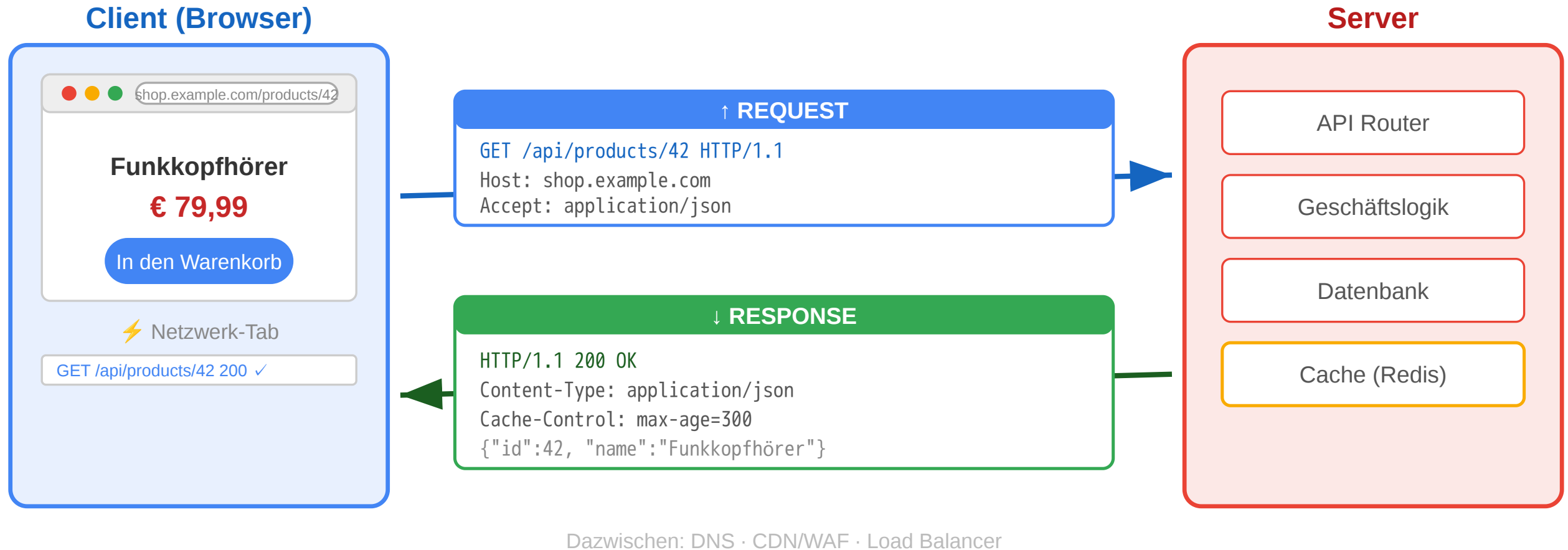
**Aufgabe:** Was ist Startzeile, Header, Body? Welche Information steckt in jedem Header?

| Teil       | Request                       | Response                          |
|------------|-------------------------------|-----------------------------------|
| Startzeile | GET /api/products/42 HTTP/1.1 | HTTP/1.1 200 OK                   |
| Header     | Host, Accept, Cookie          | Content-Type, Cache-Control, ETag |
| Body       | kein Body bei GET             | JSON-Daten des Produkts           |

**Cache-Control: max-age=300**  
Daten 5 min gültig  
**ETag: "f7e3a1b2"**  
Fingerprint zur Validierung



# HTTP: Das Request-Response-Protokoll



# Von der Fetch API zum Protokoll

In Vorlesung 03 haben wir die **Client-Seite** gesehen:

```
const response = await fetch('/api/products/42', {  
  headers: { 'Accept': 'application/json' }  
});
```

Jetzt schauen wir auf die **Protokollseite**:

| Fetch API                       | HTTP-Protokoll                                          |
|---------------------------------|---------------------------------------------------------|
| <code>fetch(url)</code>         | Browser erzeugt HTTP-Request mit Startzeile + Headern   |
| <code>{ method: 'POST' }</code> | HTTP-Methode — definiert Absicht und Semantik           |
| <code>{ headers: {...} }</code> | Request-Header — Metadaten für Server und Infrastruktur |
| <code>response.status</code>    | Statuscode — standardisiertes Ergebnis                  |
| <code>response.json()</code>    | Response Body parsen — Repräsentation der Ressource     |

Die Fetch API ist eine **JavaScript-Abstraktion über HTTP**. Wer HTTP versteht, versteht auch, warum fetch so funktioniert wie es funktioniert — und kann Netzwerkprobleme im DevTools-Netzwerk-Tab

diagnostizieren.

# HTTP-Designprinzipien

| Prinzip              | Bedeutung                                  | Konsequenz                                                |
|----------------------|--------------------------------------------|-----------------------------------------------------------|
| Ressourcenorientiert | Jede Information hat eine Adresse (URI)    | Einheitliche Adressierung weltweit                        |
| Request-Response     | Client fragt, Server antwortet             | Klare Rollenverteilung                                    |
| Zustandslos          | Jeder Request ist unabhängig               | Server muss keinen Kontext halten → skaliert              |
| Erweiterbar          | Header erlauben beliebige Metadaten        | Caching, Auth, Content-Negotiation ohne Protokolländerung |
| Textbasiert          | Nachrichten sind menschenlesbar (HTTP/1.1) | Einfach zu debuggen                                       |

## Warum das skaliert

- Zustandslosigkeit erlaubt Load Balancing über beliebig viele Server
- Proxies und Caches können HTTP-Nachrichten ohne Anwendungswissen verarbeiten
- Klare Trennung von Transport und Anwendungslogik

## URI (Uniform Resource Identifier)

`https://shop.example.com:443/api/products/42?lang=de#details`

Schema → Host → Port → Pfad → Query → Fragment



# HTTP-Methoden als semantischer Vertrag

| Methode | Absicht                          | Sicher? | Idempotent? | Online-Shop-Beispiel  |
|---------|----------------------------------|---------|-------------|-----------------------|
| GET     | Ressource lesen                  | ja      | ja          | Produktseite anzeigen |
| POST    | Neue Ressource oder Verarbeitung | nein    | nein        | Bestellung aufgeben   |
| PUT     | Ressource vollständig ersetzen   | nein    | ja          | Profil aktualisieren  |
| PATCH   | Ressource teilweise ändern       | nein    | nein*       | Adresse ändern        |
| DELETE  | Ressource entfernen              | nein    | ja          | Konto löschen         |
| HEAD    | Nur Header, kein Body            | ja      | ja          | Existenzprüfung       |
| OPTIONS | Erlaubte Methoden abfragen       | ja      | ja          | CORS Preflight        |

**Sicher (Safe):** Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.

**Idempotent:** Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

# HTTP-Methoden: Safe und Idempotent in der Praxis

**Szenario:** Ein Nutzer klickt zweimal schnell auf „Jetzt kaufen“. Netzwerk-Lag verzögert den ersten Request, beide kommen beim Server an.

| Methode               | Effekt beim Doppel-Request                      | Warum?                                                 |
|-----------------------|-------------------------------------------------|--------------------------------------------------------|
| POST /api/orders      | <b>Zwei Bestellungen</b> entstehen              | Nicht idempotent — jeder Aufruf erzeugt neue Ressource |
| PUT /api/cart/item/42 | <b>Eine Änderung</b> — zweiter Call ohne Effekt | Idempotent — gleiches Ergebnis egal wie oft            |

**Konsequenz:** Load Balancer und API-Gateways können idempotente Requests bei Timeout sicher wiederholen. CDNs cachen nur **safe** Methoden (GET, HEAD), weil diese die Ressource nie verändern.

# HTTP-Statuscodes

| Klasse | Bedeutung     | Wichtige Codes                                                                         |
|--------|---------------|----------------------------------------------------------------------------------------|
| 1xx    | Information   | 101 Switching Protocols                                                                |
| 2xx    | Erfolg        | 200 OK, 201 Created, 204 No Content                                                    |
| 3xx    | Umleitung     | 301 Moved Permanently, 304 Not Modified                                                |
| 4xx    | Client-Fehler | 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests |
| 5xx    | Server-Fehler | 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable                    |

## Was stimmt an dieser API-Antwort nicht?

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "error": "Produkt nicht gefunden"
}
```

### Problem: Statuscode lügt

- 200 OK signalisiert Erfolg
- Body enthält aber einen Fehler
- CDNs cachen die „Erfolg“-Antwort
- Monitoring zählt 200 als OK

**Richtig:** 404 Not Found als Statuscode, Fehlerdetails im Body.

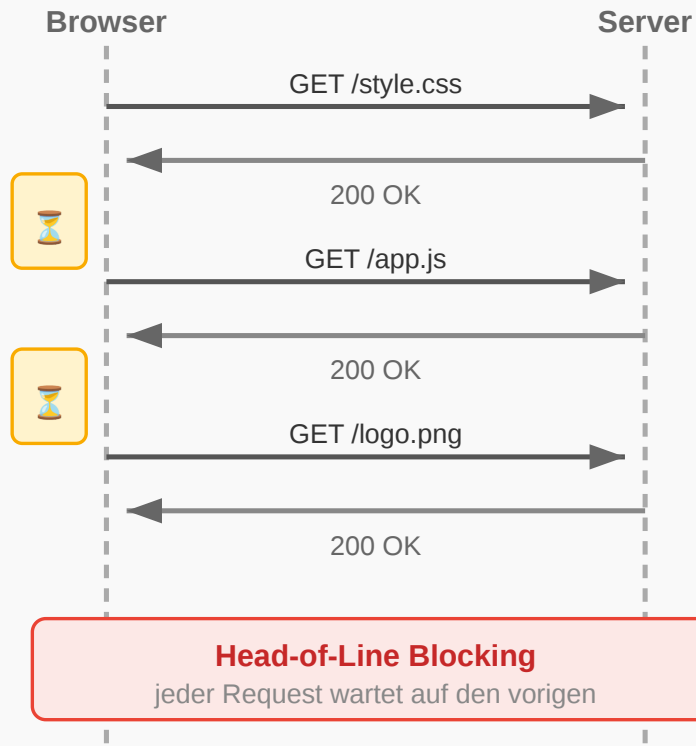
# HTTP-Versionen im Vergleich

| Version  | Jahr | Kernverbesserung                        |
|----------|------|-----------------------------------------|
| HTTP/1.0 | 1996 | Ein Request pro TCP-Verbindung          |
| HTTP/1.1 | 1997 | Persistent Connections, Pipelining      |
| HTTP/2   | 2015 | Multiplexing, Header-Kompression, binär |
| HTTP/3   | 2022 | QUIC (UDP-basiert), schnellerer Aufbau  |



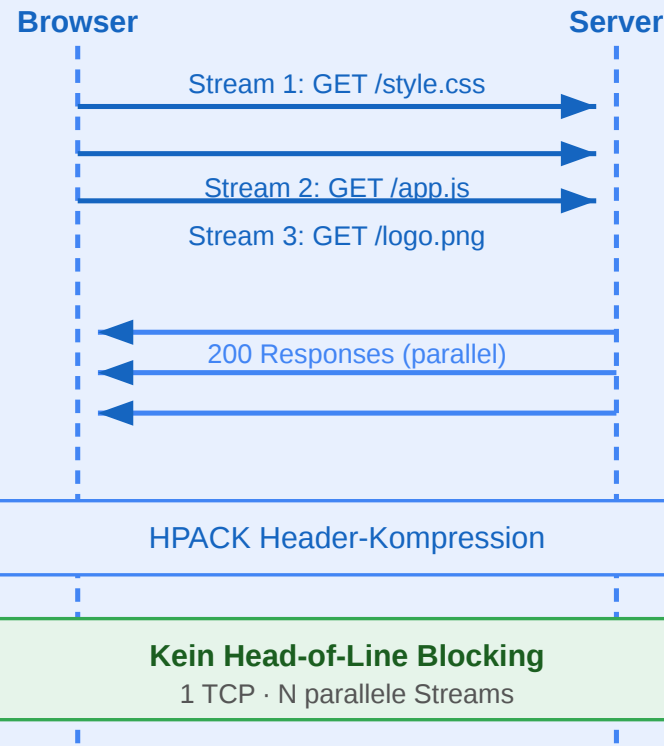
## HTTP/1.1

Sequentiell · TCP



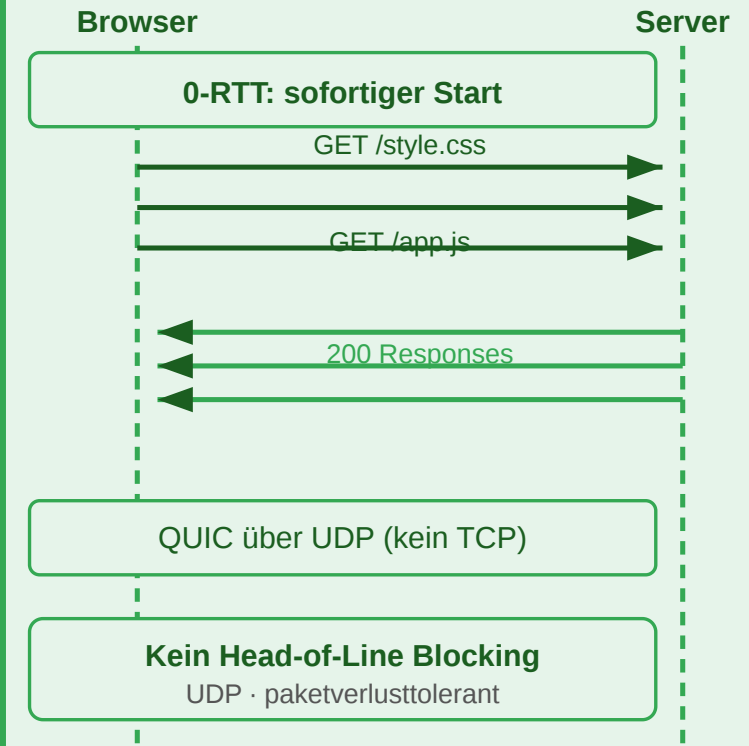
## HTTP/2

Multiplexing · TCP



## HTTP/3

QUIC · UDP · 0-RTT



**Frage:** Eine Produktseite lädt 60 Ressourcen (JS, CSS, Bilder). Warum dauert das mit HTTP/1.1 deutlich länger als mit HTTP/2 — obwohl die Bandbreite dieselbe ist?

HTTP ist nicht nur Datentransport — es ist das **Interaktionsmodell** des Webs. Methoden definieren Absichten, Statuscodes kommunizieren Ergebnisse, Header steuern Verhalten. Infrastruktur (CDN, Load Balancer, WAF) verlässt

# HTTP Header und Repräsentationen

---

**Leitfrage:** Wie beschreiben HTTP-Header, welches Format gesendet wird, wie es interpretiert werden soll und welche Varianten einer Ressource ausgehandelt werden?

# HTTP-Header im Überblick

Eine HTTP-Nachricht besteht aus:

## 1. Startzeile

Request: Methode + Ziel + Version

Response: Version + Statuscode + Statusmeldung

## 2. Header-Felder

Metadaten als Name: Wert

## 3. Leerzeile

Trennt Header und Body

## 4. Optionaler Body

Die eigentliche Repräsentation

```
GET /products/42 HTTP/1.1  
Host: shop.example.com  
Accept: application/json
```

# Wie sind HTTP-Header strukturiert?

| Teil        | Bedeutung                    | Beispiel         |
|-------------|------------------------------|------------------|
| Feldname    | Standardisierter Header-Name | Content-Type     |
| Doppelpunkt | Trennt Name und Wert         | :                |
| Feldwert    | Semantik des Headers         | application/json |

```
Content-Type: application/json; charset=utf-8
Cache-Control: public, max-age=3600
Authorization: Bearer eyJhbGciOi...
```

- Header-Namen sind **case-insensitive**
- Ein Feldwert kann aus **Token, Parametern oder Listen** bestehen
- Viele Header haben eine klar definierte **Syntax nach RFC**



# Kategorien von Header-Feldern

| Kategorie              | Typische Header                                                  | Zweck                                    |
|------------------------|------------------------------------------------------------------|------------------------------------------|
| Request-Header         | Host, Accept, Authorization, Cookie                              | Beschreiben den eingehenden Request      |
| Response-Header        | Server, Location, Set-Cookie                                     | Beschreiben Eigenschaften der Antwort    |
| Repräsentations-Header | Content-Type, Content-Length, Content-Encoding, Content-Language | Beschreiben den Body                     |
| Caching / Conditional  | Cache-Control, ETag, If-None-Match, Last-Modified                | Steuern Wiederverwendung und Validierung |
| Sicherheits-Header     | Strict-Transport-Security, Content-Security-Policy               | Erhöhen Sicherheitseigenschaften         |

**Wichtig:** Dieselbe HTTP-Ressource kann mehrere Repräsentationen haben. Die Header beschreiben, **welche Variante gerade übertragen wird**.

# Typische Request- und Response-Header

| Header        | Typ      | Kurz erklärt                                                                      |
|---------------|----------|-----------------------------------------------------------------------------------|
| Host          | Request  | Gibt an, fuer welchen Host der Request bestimmt ist; wichtig bei virtuellen Hosts |
| Authorization | Request  | Uebertraegt Anmeldedaten oder Zugriffstoken                                       |
| Location      | Response | Verweist auf die URI einer neuen oder anderen Ressource                           |
| Server        | Response | Kann Informationen ueber die Server-Software enthalten                            |

```
POST /login HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJ...
```

# Typische Header für Repräsentation und Kontrolle

| Header                    | Kategorie      | Kurz erklärt                                                    |
|---------------------------|----------------|-----------------------------------------------------------------|
| Content-Length            | Repräsentation | Nennt die Länge des Bodys in Bytes                              |
| Content-Encoding          | Repräsentation | Gibt an, ob der Body z. B. mit gzip komprimiert wurde           |
| Content-Security-Policy   | Sicherheit     | Beschränkt, welche Inhalte eine Seite laden oder ausführen darf |
| Strict-Transport-Security | Sicherheit     | Erzwingt für eine Domain die Nutzung von HTTPS                  |

## Merksatz:

Ein Teil der Header beschreibt den **Body**, ein anderer Teil steuert **Verhalten** von Clients, Caches und Browsern.

# Medienrepräsentationen und MIME Types

Eine Ressource ist nicht gleich ihr Datenformat. Dieselbe Ressource kann als verschiedene **Medientypen** übertragen werden:

| Medientyp              | Bedeutung       | Beispiel                |
|------------------------|-----------------|-------------------------|
| text/html              | HTML-Dokument   | Produktseite im Browser |
| application/json       | JSON-Daten      | API-Response            |
| text/css               | Stylesheet      | Gestaltung einer Seite  |
| application/javascript | JavaScript-Code | Skriptdatei             |
| image/png              | PNG-Bild        | Produktfoto             |
| application/pdf        | PDF-Dokument    | Rechnung zum Download   |

**Media Type = Typ/Subtyp**

Beispiele: text/html, application/json, image/svg+xml

Der Medientyp beschreibt nicht die Ressource selbst, sondern die Form ihrer aktuellen Repräsentation.

# Verschiedene Clients, verschiedene Repräsentationen

Nicht jeder Client braucht dieselbe Darstellung derselben Ressource:

| Client-Modell                     | Typische Repräsentation    | Wofür geeignet?                                   |
|-----------------------------------|----------------------------|---------------------------------------------------|
| Server-rendererter Browser-Client | text/html                  | Der Server liefert direkt eine fertige HTML-Seite |
| JavaScript-Frontend / SPA         | application/json           | Das Frontend rendert die Oberfläche selbst        |
| Mobile App                        | application/json           | Die App verarbeitet Daten und baut die UI nativ   |
| Download-Client                   | application/pdf, image/png | Datei- oder Medienaussgabe                        |

## Kernidee:

Die fachliche Ressource bleibt gleich, aber die **Repräsentation** wird an den Bedarf des Clients angepasst.

# HTML vs. JSON als Repräsentation

## HTML-Repräsentation

- Enthält Struktur und Inhalt fuer die direkte Anzeige im Browser
- Typisch fuer **serverseitig gerenderte** Anwendungen
- Media Type: text/html

## JSON-Repräsentation

- Enthält strukturierte Daten statt fertiger Seiten
- Typisch fuer **APIs**, SPAs und mobile Clients
- Media Type: application/json

## Wichtig:

HTML und JSON konkurrieren nicht zwingend. Beide koennen sinnvolle Repräsentationen **derselben Ressource** sein.

# JSON kurz eingeführt

**JSON** steht fuer **JavaScript Object Notation**. Es ist ein leichtgewichtiges Textformat fuer strukturierte Daten.

```
{  
  "id": 42,  
  "name": "Funkkopfhörer",  
  "price": 79.99,  
  "in_stock": true  
}
```

| JSON-Element | Beispiel     |
|--------------|--------------|
| Objekt       | { "id": 42 } |
| Array        | [1, 2, 3]    |
| String       | "name"       |
| Zahl         | 79.99        |
| Boolean      | true, false  |
| Null         | null         |

# Content Negotiation: Grundidee

| Header           | Richtung              | Bedeutung                                        |
|------------------|-----------------------|--------------------------------------------------|
| Content-Type     | Request oder Response | Welches Format der Body hat                      |
| Accept           | Request               | Welche Formate der Client akzeptiert             |
| Accept-Language  | Request               | Bevorzugte Sprache                               |
| Accept-Encoding  | Request               | Bevorzugte Kompression, z. B. gzip oder br       |
| Content-Encoding | Response              | Welche Kompression auf den Body angewendet wurde |

```
GET /products/42 HTTP/1.1
Accept: application/json
Accept-Language: de
Accept-Encoding: gzip, br
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Language: de
Content-Encoding: gzip
```



# Content Negotiation: Repräsentation auswählen

Ein Client kann mehrere Formate akzeptieren und Prioritäten angeben:



| Teil                   | Bedeutung                                          |
|------------------------|----------------------------------------------------|
| text/html              | Bevorzugte Repräsentation                          |
| application/json;q=0.9 | Ebenfalls akzeptabel, aber etwas weniger bevorzugt |

| Teil                  | Bedeutung              |
|-----------------------|------------------------|
| application/xml;q=0.5 | Nur niedrige Priorität |

## q-Wert = Qualitätswert

Je hoeher der q-Wert, desto staerker die Präferenz des Clients.

Wenn kein gemeinsames Format gefunden wird, kann der Server z. B. mit 406 Not Acceptable antworten.

# Eine Ressource, mehrere Repräsentationen

Beispiel: Dieselbe Produkt-Ressource /products/42

## Browser-orientiert

```
GET /products/42 HTTP/1.1  
Accept: text/html  
  
HTTP/1.1 200 OK  
Content-Type: text/html
```

## API-orientiert

```
GET /products/42 HTTP/1.1  
Accept: application/json  
  
HTTP/1.1 200 OK  
Content-Type: application/json
```

## Kernidee:

Die **Ressource** bleibt dieselbe, aber ihre **Repräsentation** kann je nach Client unterschiedlich sein.

# Warum das für HTTP-Verständnis wichtig ist

- Content-Type und Accept machen Repräsentationen explizit
- Caches müssen wissen, **welche Variante** einer Ressource gespeichert wurde
- Infrastruktur kann nur korrekt arbeiten, wenn Header die Nachricht eindeutig beschreiben
- Dieselbe fachliche Ressource kann für unterschiedliche Clients unterschiedlich dargestellt werden

Header sind nicht Beiwerk, sondern Teil der HTTP-Semantik. Sie beschreiben Formate, Aushandlung, Kompression, Caching, Sicherheit und Zustand. Ohne dieses Verständnis bleiben spätere Themen wie Content Negotiation, Cache-Control oder API-Design unvollständig.

# HTTP Kontrolle, Caching und Sicherheit

---

**Leitfrage:** Wie bleibt HTTP trotz Zustandslosigkeit effizient und steuerbar, wenn Inhalte umgeleitet, zwischengespeichert oder erneut validiert werden müssen?

# Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von € 79,99 auf € 69,99 **gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

**Aufgabe:** Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Der Browser hat die vorherige Response gecacht. Der Cache-Control: max-age=300 Header sagte: "Diese Daten sind 5 Minuten gueltig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

**Architekturentscheidung:** Wie lange sollen Produktdaten gecacht werden? Zu lang -> veraltete Preise. Zu kurz -> jeder Seitenaufruf belastet den Server.

# Caching: Ebenen und Mechanismen

| Cache-Ebene       | Wo?                          | Vorteil                                      |
|-------------------|------------------------------|----------------------------------------------|
| Browser-Cache     | Lokal auf dem Gerät          | Kein Netzwerk-Request -> schnellste Variante |
| Proxy/CDN-Cache   | Edge-Server im Netzwerk      | Reduziert Last auf Origin, niedrige Latenz   |
| Application-Cache | Im Server (Redis, Memcached) | Vermeidet wiederholte Datenbankabfragen      |

Jede Ebene spart Arbeit an einer anderen Stelle:

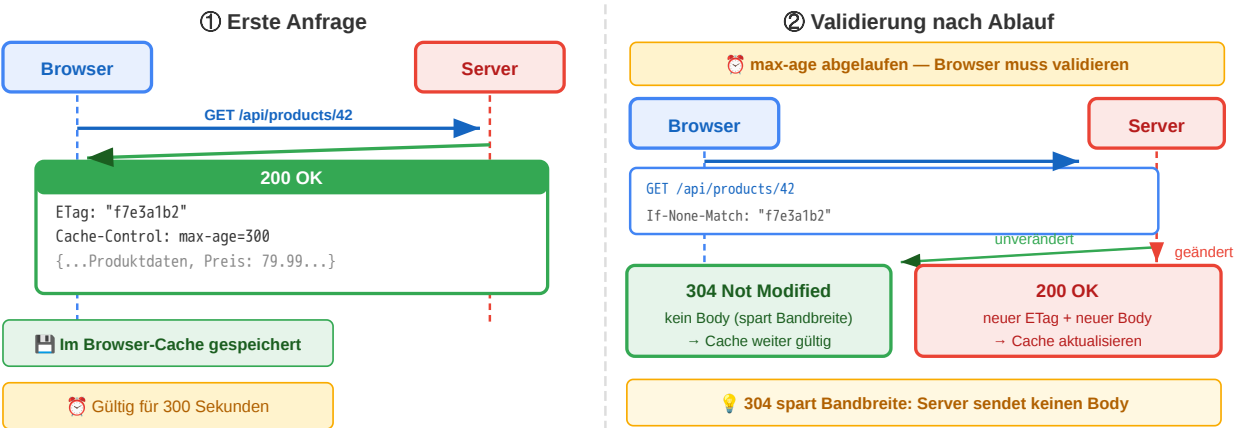
- Browser-Cache spart Netzwerkverkehr
- CDN spart Origin-Last
- Application-Cache spart Datenbank- oder Rechenaufwand

# Cache-Steuerung über Header

| Header                      | Funktion                   | Beispiel             |
|-----------------------------|----------------------------|----------------------|
| Cache-Control: max-age=3600 | 1 Stunde gültig            | Statische Assets     |
| Cache-Control: no-cache     | Immer validieren           | Produktdaten, Preise |
| Cache-Control: no-store     | Nie cachen                 | Bankdaten            |
| Cache-Control: public       | CDN darf cachen            | Öffentliche Inhalte  |
| Cache-Control: private      | Nur Browser                | Nutzerdaten          |
| ETag                        | Ressourcen-Fingerprint     | "a1b2c3d4"           |
| Last-Modified               | Letzter Änderungszeitpunkt | Mon, 23 Mar 2026     |

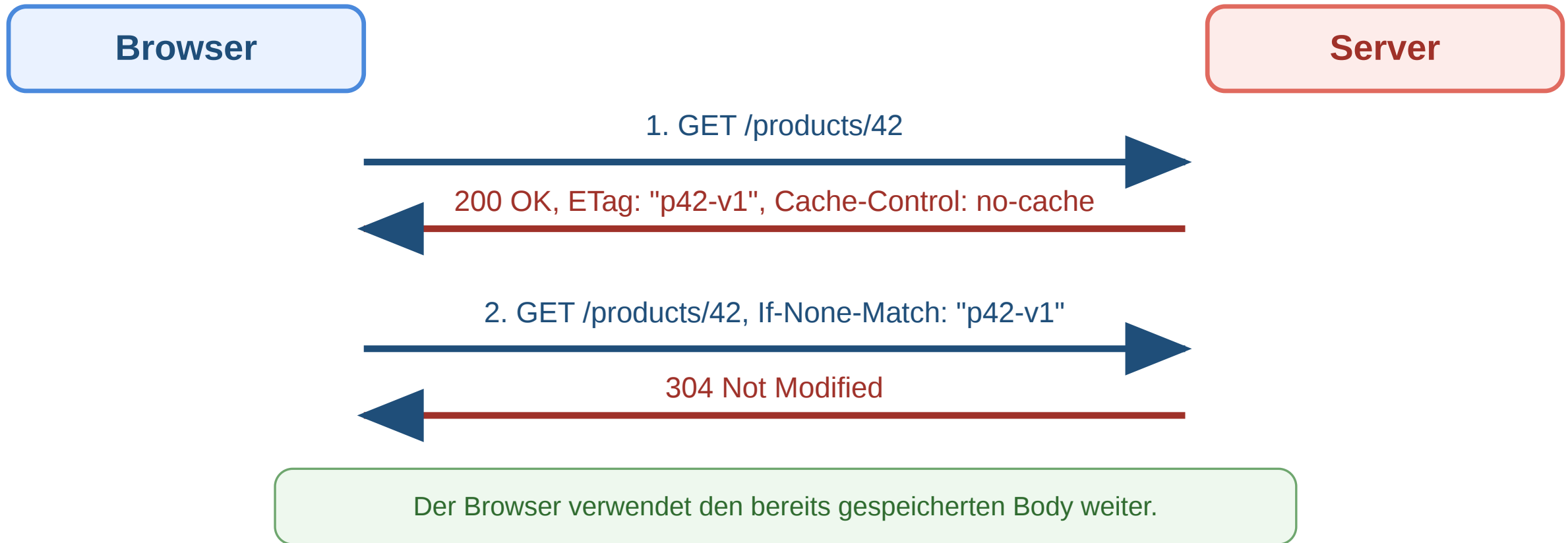
**Wie der Browser entscheidet:** Cache vorhanden? → max-age abgelaufen? → Validierungsrequest mit If-None-Match: ETag

**Validierung:** Server antwortet 304 Not Modified (kein Body, spart Bandbreite) oder 200 OK + neuer Body + neuer ETag.





# Workflow: Conditional Request und Cache-Validierung



| Schritt              | Bedeutung                               |
|----------------------|-----------------------------------------|
| ETag empfangen       | Client kennt Version der Repräsentation |
| If-None-Match senden | Client fragt: Hat sich etwas geändert?  |

| Schritt          | Bedeutung                                  |
|------------------|--------------------------------------------|
| 304 Not Modified | Kein neuer Body nötig, Cache bleibt gültig |

# Interpretationsaufgabe: Caching-Strategie

**Aufgabe:** Welchen Cache-Control-Header würden Sie für diese Ressourcen des Online-Shops setzen?

| Ressource                                      | Ihr Vorschlag? |
|------------------------------------------------|----------------|
| app.v3.2.1.js (versionierte JS-Datei)          |                |
| /api/products/42 (Produktdaten mit Preis)      |                |
| /api/cart (Warenkorb des eingeloggten Nutzers) |                |
| logo.png (Shop-Logo)                           |                |

| Ressource        | Header                              | Begründung                                   |
|------------------|-------------------------------------|----------------------------------------------|
| app.v3.2.1.js    | public, max-age=31536000, immutable | Version im Dateinamen → ändert sich nie      |
| /api/products/42 | public, no-cache + ETag             | Preise können sich ändern, Validierung nötig |
| /api/cart        | private, no-store                   | Nutzerspezifisch, darf nicht im CDN landen   |
| logo.png         | public, max-age=86400               | Ändert sich selten, 1 Tag reicht             |

# Redirects und das Post/Redirect/Get-(PRG)-Pattern

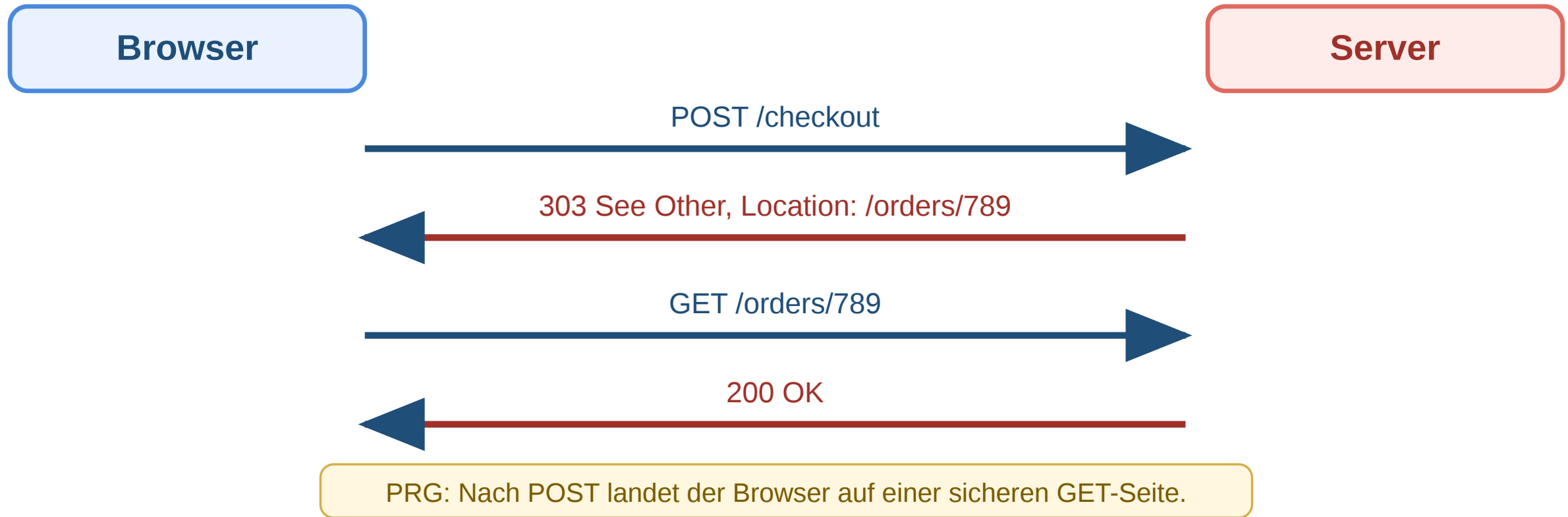
```
Browser — POST /api/orders —> Server
Server — 303 See Other —> Browser
Browser — GET /orders/789 —> Server
Server — 200 OK (Bestellbestätigung) —> Browser
```

| Code    | Semantik                  | Typischer Einsatz              |
|---------|---------------------------|--------------------------------|
| 301     | Permanent verschoben      | Domain-Umzug                   |
| 302/307 | Temporär woanders         | A/B-Test, Wartung              |
| 303     | Nach POST auf GET         | <b>Post/Redirect/Get (PRG)</b> |
| 308     | Permanent, Methode bleibt | API-Migration                  |

## Post/Redirect/Get (PRG) verhindert doppelte

**Bestellungen:** Nach POST antwortet der Server mit 303 See Other → der Browser folgt per GET → Neuladen der Seite wiederholt nur den GET, nicht den POST.

# Workflow: Redirects in der Praxis

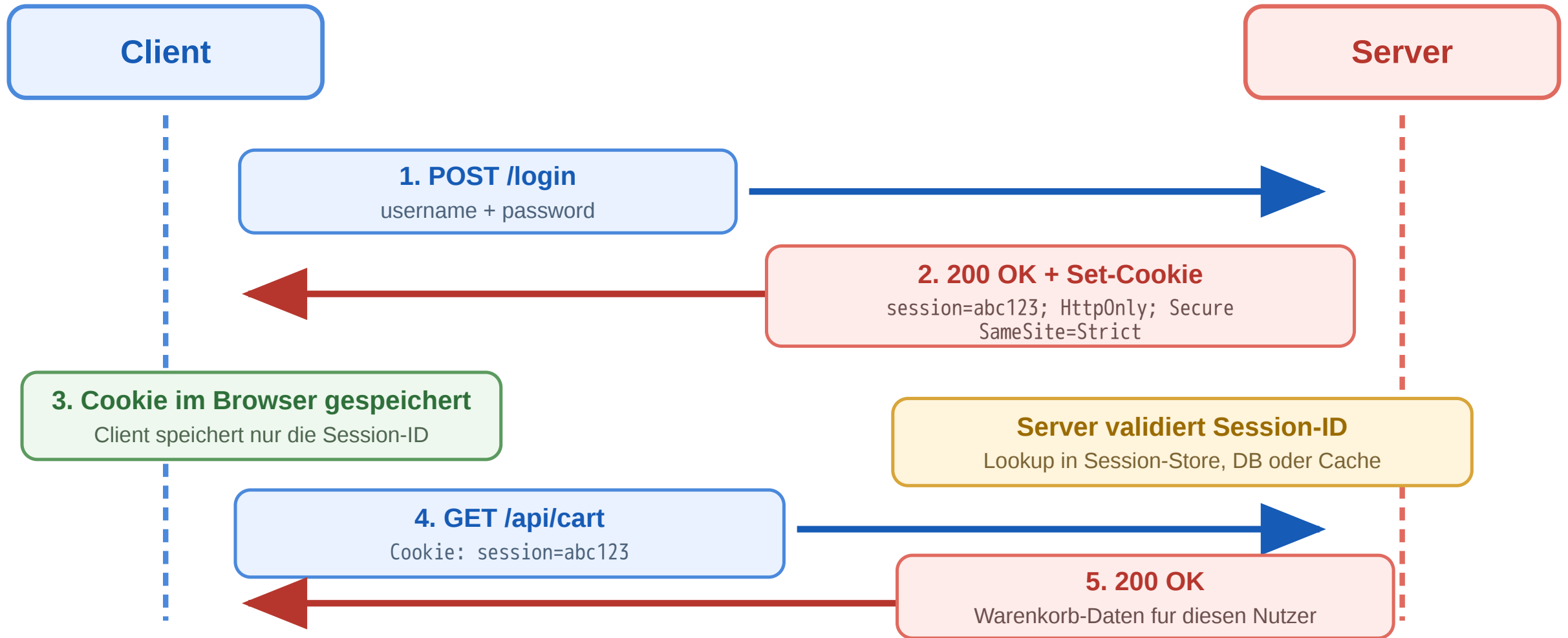


| Status | Typischer Effekt                              |
|--------|-----------------------------------------------|
| 301    | Dauerhaft verschoben                          |
| 302    | Temporär verschoben, historisch uneinheitlich |
| 303    | Nach POST per GET weiter                      |

| Status  | Typischer Effekt                  |
|---------|-----------------------------------|
| 307/308 | Methode und Body bleiben erhalten |

# Sessions und Cookies

HTTP ist zustandslos — aber der Online-Shop braucht Login und Warenkorb. **Cookies** lösen diesen Widerspruch:



 Welche Cookie-Konfiguration ist sicher?

| Konfiguration                                     | XSS-sicher? | CSRF-sicher? | HTTPS-only? |
|---------------------------------------------------|-------------|--------------|-------------|
| session=abc123                                    | nein        | nein         | nein        |
| session=abc123; HttpOnly                          | <b>ja</b>   | nein         | nein        |
| session=abc123; HttpOnly; Secure                  | <b>ja</b>   | nein         | <b>ja</b>   |
| session=abc123; HttpOnly; Secure; SameSite=Strict | <b>ja</b>   | <b>ja</b>    | <b>ja</b>   |

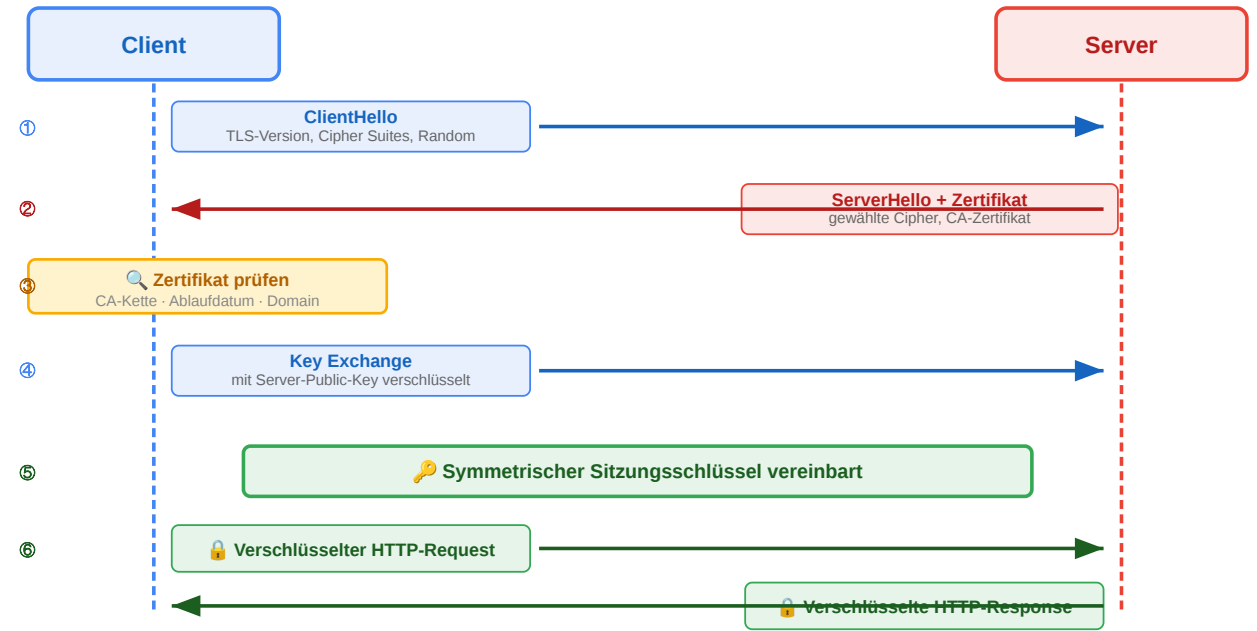
Nur die letzte Variante schützt gegen die drei häufigsten Angriffsvektoren.



# HTTPS und TLS

| Eigenschaft     | HTTP            | HTTPS                |
|-----------------|-----------------|----------------------|
| Vertraulichkeit | Klartext lesbar | Verschlüsselt        |
| Integrität      | Manipulierbar   | Manipulation erkannt |
| Authentizität   | Keine Garantie  | Zertifikat bestätigt |

**Diskussionsfrage:** Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Welche der drei TLS-Eigenschaften schützt hier — und welche nicht?  
(Hinweis: TLS schützt den Transport, nicht den Endpunkt.)



HTTP bleibt simpel auf Protokollebene, wird aber durch Header und Statuscodes zum ausdrucksstarken Steuerungsmechanismus. Caching ist der wichtigste Performance-Hebel — aber jede Caching-Entscheidung ist ein Trade-off zwischen Aktualität und Last. Redirects steuern Navigation, PRG verhindert Doppelverarbeitung. Cookies ermöglichen Zustand trotz Zustandslosigkeit — aber nur mit korrekter Konfiguration. TLS sichert den Transport.

# REST als Architekturidee

---

**Leitfrage:** Was unterscheidet eine RESTful API von einem beliebigen HTTP-Endpunkt, der nur JSON ausliefert?

# REST kurz eingeführt

REST steht fuer **Representational State Transfer**.

REST entstand nicht als Framework oder Produkt, sondern als Beschreibung dafuer, **wie das Web bereits erfolgreich funktioniert**:

- Ressourcen haben stabile Adressen (/products/42)
- Clients navigieren ueber standardisierte HTTP-Operationen
- Server liefern **Repräsentationen** dieser Ressourcen
- Infrastruktur wie Browser, Caches und Proxies kann mitarbeiten

## Entstehungskontext

- Ende der 1990er Jahre im Umfeld der Web-Architektur
- Geprägt durch **Roy Fielding**
- Ausgangsfrage: Warum skaliert das Web trotz seiner Einfachheit so gut?

**Fuer die Praxis:** REST ist zuerst ein **Programmier- und Entwurfsparadigma fuer verteilte Client-Server-Systeme**. Die formale Einordnung als Architekturstil kommt im naechsten Schritt.

# Eine typische AI-Antwort auf diese Frage

*„REST ist ein Architekturstil für APIs, bei dem man JSON über HTTP verschickt. Man benutzt GET zum Lesen, POST zum Erstellen, PUT zum Aktualisieren und DELETE zum Löschen. Die URLs sollten Ressourcen beschreiben, z.B. /users/42.“*

**Aufgabe:** Was ist an dieser Antwort korrekt? Was fehlt? Was ist irreführend?

## Analyse

| Aussage                                | Bewertung                                                                                        |
|----------------------------------------|--------------------------------------------------------------------------------------------------|
| „JSON über HTTP“                       | <b>Irreführend</b> — REST ist formatunabhängig. JSON ist verbreitet, aber nicht Voraussetzung.   |
| „GET zum Lesen, POST zum Erstellen...“ | <b>Teilweise korrekt</b> — beschreibt CRUD-Mapping, aber nicht die eigentlichen REST-Constraints |
| „URLs sollten Ressourcen beschreiben“  | <b>Korrekt</b> — aber nur ein Aspekt der Uniform Interface                                       |
| Fehlt: Statelessness                   | <b>Kritisch</b> — Zustandslosigkeit ist der Grund, warum REST skaliert                           |
| Fehlt: Cacheability                    | <b>Kritisch</b> — ein Hauptvorteil gegenüber RPC-Ansätzen                                        |
| Fehlt: Layered System                  | <b>Wichtig</b> — ermöglicht CDN, Load Balancer, API Gateway transparent einfügbar                |

Die AI-Antwort beschreibt die **Syntax** von REST, aber nicht die **Architektur**. Das ist der häufigste Fehler.

# REST: Ein Architekturstil, kein Protokoll

| Constraint                | Bedeutung                                              | Konsequenz                                                  |
|---------------------------|--------------------------------------------------------|-------------------------------------------------------------|
| Client-Server             | Klare Rollentrennung                                   | Client und Server entwickeln sich unabhängig                |
| Stateless                 | Jeder Request enthält alle nötigen Informationen       | Server skaliert horizontal, kein Session-Zustand            |
| Cacheable                 | Responses deklarieren ihre Cachebarkeit                | Infrastruktur kann Caching transparent umsetzen             |
| Uniform Interface         | Einheitliche Schnittstelle für alle Ressourcen         | Generische Werkzeuge (Browser, curl, Proxies) funktionieren |
| Layered System            | Client weiß nicht, ob er direkt mit dem Server spricht | Load Balancer, CDN, API Gateway transparent einfügbar       |
| Code on Demand (optional) | Server kann ausführbaren Code liefern                  | JavaScript im Browser                                       |

## Kernidee

REST nutzt die **vorhandene Web-Infrastruktur** (HTTP, URIs, Caching, Proxies) als Anwendungsplattform — statt ein eigenes RPC-Protokoll darüber zu bauen.

# Welchen Constraint verletzt dieses Design?

**Szenario 1:** Der Online-Shop speichert den Warenkorb in einer serverseitigen Session. Beim nächsten Request muss der Client denselben Server treffen (Sticky Sessions).

**Szenario 2:** Die API liefert bei GET /api/products keinen Cache-Control-Header. Jeder Request geht bis zum Origin-Server.

**Szenario 3:** Der Client kennt die interne Datenbankstruktur und baut SQL-artige Queries: GET /api/query?table=products&where=price>50

| Szenario              | Verletzter Constraint                     | Konsequenz                                                              |
|-----------------------|-------------------------------------------|-------------------------------------------------------------------------|
| 1: Sticky Sessions    | <b>Stateless</b>                          | Horizontale Skalierung eingeschränkt, Server-Ausfall = Session verloren |
| 2: Kein Cache-Control | <b>Cacheable</b>                          | CDN nutzlos, unnötige Serverlast, höhere Latenz                         |
| 3: SQL-artige Queries | <b>Uniform Interface + Layered System</b> | Client an Interna gekoppelt, keine Abstraktion möglich                  |

# REST vs. RPC im Vergleich

## RPC-Stil (aktionsorientiert)

```
POST /createUser  
POST /getUser  
POST /updateUser  
POST /deleteUser  
POST /getUserOrders  
POST /sendPasswordReset
```

- Jeder Endpunkt ist eine **Funktion**
- HTTP-Methode immer POST
- Infrastruktur kann nichts cachen
- Client muss Doku lesen

## REST-Stil (ressourcenorientiert)

```
POST /users → anlegen  
GET /users/42 → lesen  
PUT /users/42 → ersetzen  
DELETE /users/42 → löschen  
GET /users/42/orders → Bestellungen  
POST /password-resets → anstoßen
```

- Jeder Endpunkt ist eine **Ressource**
- HTTP-Methoden drücken Absicht aus
- GET-Requests sind cachebar
- Verhalten ist vorhersagbar

## RPC-Stil

Aktions-orientiert

POST /createUser

alles POST

POST /getUser

POST /updateUser

POST /deleteUser

vs.

## REST-Stil

Ressourcen-orientiert

POST /users

→ anlegen

GET /users/42

→ lesen

PUT /users/42

→ ersetzen

DELETE /users/42

→ löschen

- ✗ CDN kann nichts cachen
- ✗ Monitoring sieht nur POST
- ✗ WAF kann nicht filtern
- ✗ Client muss Doku lesen
- ✗ Jeder Endpunkt ist eine Funktion

- ✓ GET ist cachebar (CDN)
- ✓ Methoden in Audit-Logs sichtbar
- ✓ WAF erkennt DELETE, POST
- ✓ Verhalten ist vorhersagbar
- ✓ Jeder Endpunkt ist eine Ressource

Warum das für Infrastruktur relevant ist





# Ressourcen und Repräsentationen

Eine **Ressource** ist ein fachliches Konzept. Was der Client empfaengt, ist eine **Repräsentation**.

| Konzept          | Bedeutung                                                           |
|------------------|---------------------------------------------------------------------|
| Ressource        | Das fachliche Ding (existiert im System)                            |
| URI              | Die Adresse der Ressource (identifiziert sie eindeutig)             |
| Repräsentation   | Die konkrete Darstellung (JSON, HTML, CSV - je nach Accept-Header)  |
| Zustandsübergang | Aktion auf der Ressource (GET liest, POST erzeugt, DELETE entfernt) |

**Content Negotiation:** Client und Server einigen sich ueber Accept und Content-Type auf das Format.

Dieselbe Ressource kann also ueber dieselbe URI identifiziert werden, waehrend ihre konkrete Darstellung je nach Client unterschiedlich ausfaellt.

# Wann stößt REST an Grenzen?

| Situation                                          | REST-Problem                       | Alternative                                     |
|----------------------------------------------------|------------------------------------|-------------------------------------------------|
| Client braucht Daten aus 5 Ressourcen gleichzeitig | 5 separate Requests → Latenz       | <b>GraphQL:</b> Ein Query, ein Response         |
| Echtzeit-Updates (Chat, Live-Ticker)               | Polling ist ineffizient            | <b>WebSockets:</b> bidirektionale Verbindung    |
| Interne Microservice-Kommunikation                 | JSON-Overhead, keine Typsicherheit | <b>gRPC:</b> binäres Protokoll, Protobuf-Schema |
| Komplexe Aktionen (Batch-Operationen)              | Passt nicht in CRUD                | <b>RPC-Endpunkte</b> als Ergänzung              |

## Kein Entweder-Oder

Viele Systeme **kombinieren** Stile: REST für öffentliche APIs, GraphQL für flexible Frontends, gRPC für interne Services. Die Wahl hängt von **Nutzungsszenarien** ab — nicht von Dogma.

REST ist kein JSON-over-HTTP. Es ist ein Architekturstil mit sechs Constraints, die das Web skalierbar, cachebar und erweiterbar machen. Die Stärke liegt in der konsequenten Nutzung bestehender Web-Infrastruktur. Wer die Constraints versteht, kann beurteilen, wann REST die richtige Wahl ist und wann ein anderer Stil besser passt.

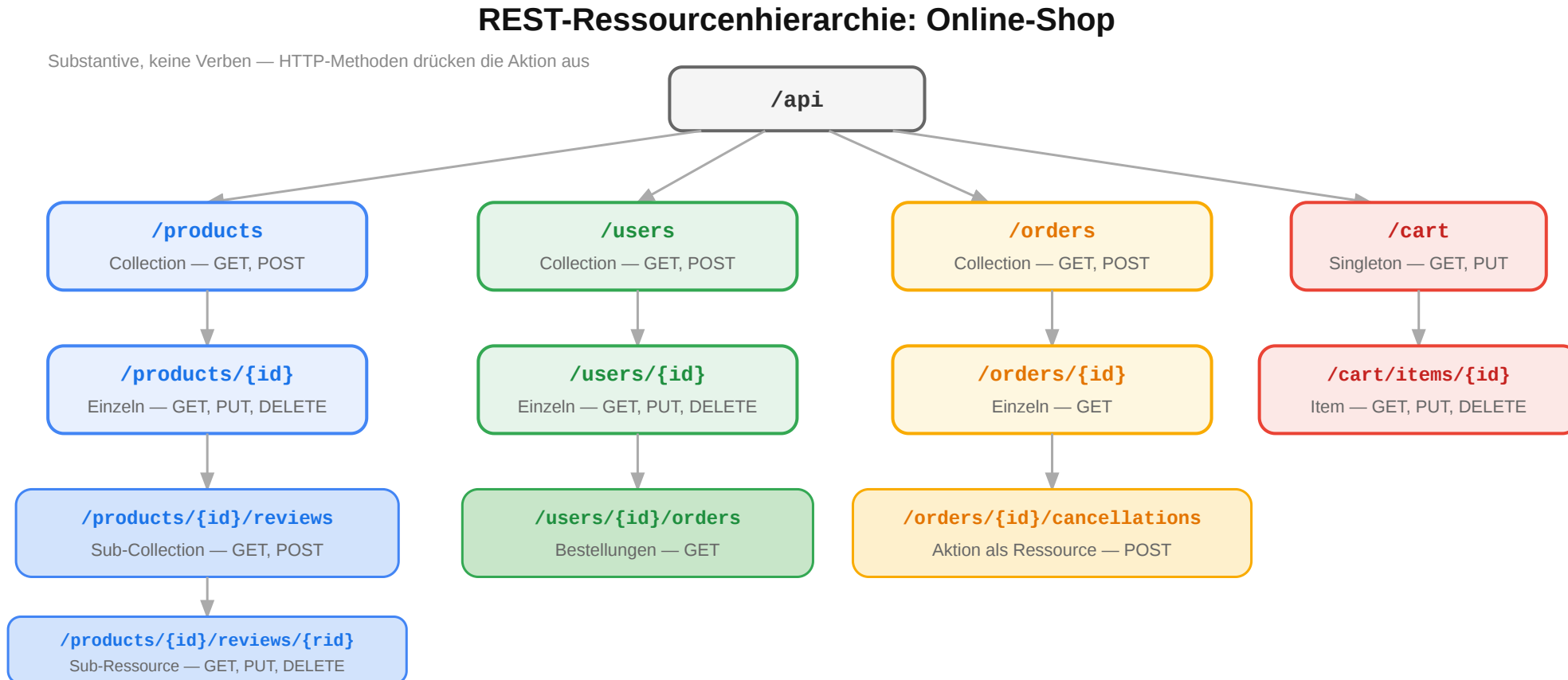
# Ressourcen und URI-Design

---

**Leitfrage:** Wie werden fachliche Objekte so als Ressourcen modelliert, dass eine API verständlich und stabil bleibt?

# Ressourcenorientiertes Design: Online-Shop

Der erste Schritt beim API-Design ist die Identifikation der **Ressourcen** — die fachlichen Dinge, mit denen Clients interagieren:



# URI-Design-Regeln

| Regel                         | Gut                  | Schlecht                 | Warum                               |
|-------------------------------|----------------------|--------------------------|-------------------------------------|
| Plural für Collections        | /products            | /product                 | Collection enthält viele            |
| Kleinschreibung, Bindestriche | /order-items         | /OrderItems              | URL-Konvention                      |
| Keine Verben im Pfad          | POST /orders         | POST /createOrder        | Methode drückt Aktion aus           |
| Hierarchie = Zugehörigkeit    | /users/42/orders     | /getUserOrders?userId=42 | Beziehung sichtbar                  |
| Query für Filter/Sortierung   | /products?sort=price | /products/sortByPrice    | Pfad = Identität, Query = Parameter |

## Anti-Patterns erkennen

```
POST /api/getUser  
    → ?  
POST /api/deleteAccount  
    → ?  
GET  /api/search?action=find → ?  
POST /api/doPayment  
    → ?
```

## Ressourcenorientierte Fassung

```
POST /api/getUser  
    → GET  /users/{id}  
POST /api/deleteAccount  
    → DELETE /accounts/{id}  
GET  /api/search?action=find → GET  
    /products?q=...  
POST /api/doPayment  
    → POST  /orders/{id}/payments
```

# Übung: API-Design für eine Domäne

**Aufgabe:** Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

## Eine mögliche Lösung

| Fachliches Konzept       | URI                             | Methoden         |
|--------------------------|---------------------------------|------------------|
| Alle Kurse               | /courses                        | GET, POST        |
| Ein Kurs                 | /courses/web-eng                | GET, PUT, DELETE |
| Vorlesungen eines Kurses | /courses/web-eng/lectures       | GET, POST        |
| Einschreibungen          | /courses/web-eng/enrollments    | GET, POST        |
| Eine Einschreibung       | /courses/web-eng/enrollments/42 | GET, DELETE      |
| Noten eines Studierenden | /students/42/grades             | GET              |
| Note in einem Kurs       | /students/42/grades/web-eng     | GET, PUT         |



**Diskussion:** Sollte die Note unter `/students/42/grades/web-eng` oder unter `/courses/web-eng/grades/42` liegen? Beide sind valide — die Wahl hängt davon ab, **wer der primäre Nutzer der API** ist.

# CRUD-Mapping auf HTTP

| Operation            | HTTP   | URI          | Body           | Response       |
|----------------------|--------|--------------|----------------|----------------|
| Liste lesen          | GET    | /products    | —              | 200 + Array    |
| Einzel lesen         | GET    | /products/42 | —              | 200 + Objekt   |
| Anlegen              | POST   | /products    | Neues Objekt   | 201 + Location |
| Vollständig ersetzen | PUT    | /products/42 | Vollständig    | 200 / 204      |
| Teilweise ändern     | PATCH  | /products/42 | Nur Änderungen | 200 / 204      |
| Löschen              | DELETE | /products/42 | —              | 204            |

## Jenseits von CRUD

Nicht alles passt in CRUD. Für **Aktionen** gibt es zwei Muster:

- **Sub-Ressource:** POST /orders/42/cancellations
- **Controller-Ressource:** POST /password-resets



# Workflow: Create / Update / Delete korrekt abbilden

## Typische Zustandsänderungen und passende Responses

**POST /products**

201 Created + Location

**PUT /products/42**

200 OK oder 204

**PATCH /products/42**

200 OK oder 204

**DELETE /products/42**

204 No Content

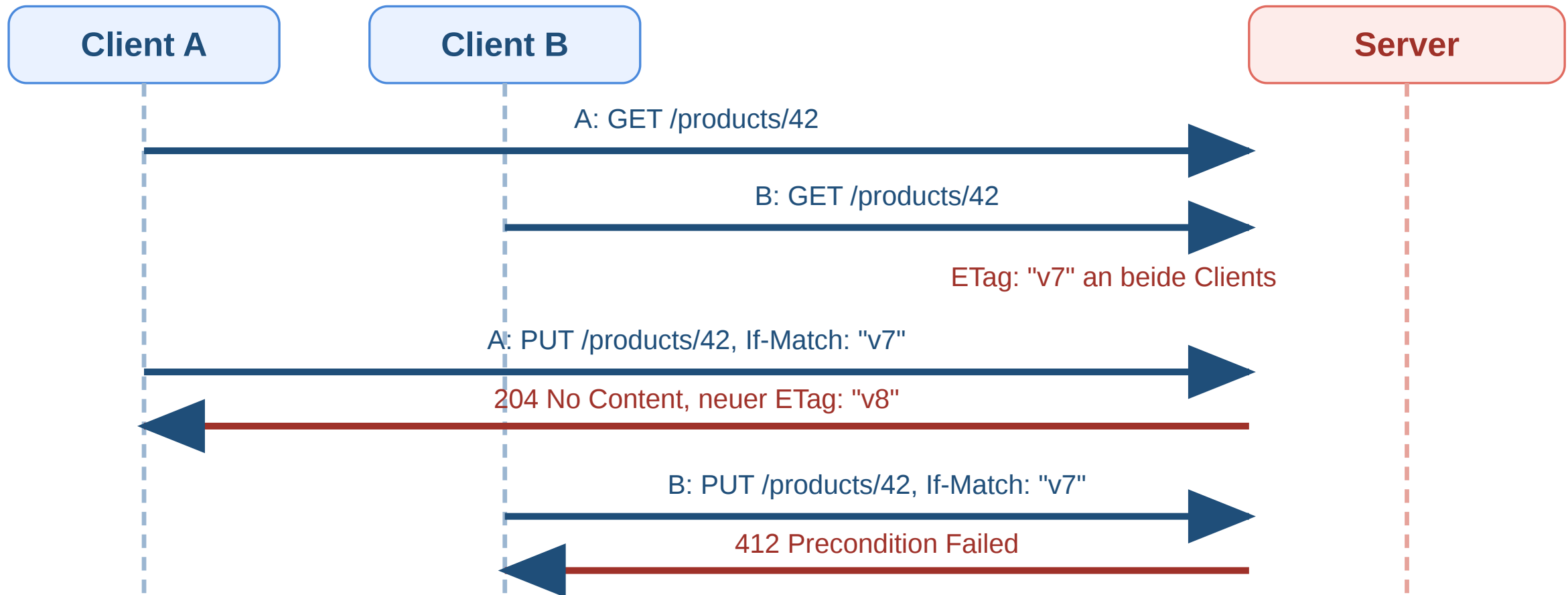
POST erzeugt typischerweise eine neue Ressource, PUT/PATCH ändern eine vorhandene, DELETE entfernt sie.  
Der Statuscode soll den fachlichen Effekt ausdrücken, nicht nur „irgendwie Erfolg“.

| Operation     | Gute Response          | Warum?                        |
|---------------|------------------------|-------------------------------|
| Anlegen       | 201 Created + Location | Neue Ressource ist entstanden |
| Aktualisieren | 200 oder 204           | Ressource existierte bereits  |



| Operation | Gute Response  | Warum?                            |
|-----------|----------------|-----------------------------------|
| Löschen   | 204 No Content | Erfolgreich, aber kein Body nötig |

# Workflow: Conditional PUT mit ETag und If-Match



**Ziel:** Das PUT wird nur ausgeführt, wenn die Ressource noch die erwartete Version hat.

# Pagination, Filter und Sortierung

GET /products?page=2&per\_page=20&sort=-price&category=electronics&q=kopfhörer

| Parameter       | Funktion                    | Beispiel                    |
|-----------------|-----------------------------|-----------------------------|
| page / per_page | Seitenbasierte Pagination   | ?page=3&per_page=25         |
| cursor          | Cursor-basierte Pagination  | ?cursor=eyJpZCI6NDJ9        |
| sort            | Sortierung (- = absteigend) | ?sort=-created_at,name      |
| filter          | Filterung                   | ?status=active&min_price=10 |
| q               | Volltextsuche               | ?q=kopfhörer                |

## Pagination in der Response

```
{
  "data": [{"id": 42, "name": "Funkkopfhörer", "price": 79.99}],
  "meta": { "total": 142, "page": 2, "per_page": 20 },
  "links": {
    "next": "/products?page=3&per_page=20",
    "prev": "/products?page=1&per_page=20"
  }
}
```

Links machen die API navigierbar — der Client muss keine URIs konstruieren.



Gute REST-APIs beginnen mit sauber geschnittenen Ressourcen. URI-Design ist eine **Modellierungsaufgabe**: Ressourcen identifizieren, Beziehungen als Hierarchie abbilden, Aktionen über HTTP-Methoden ausdrücken. Die Entscheidung, wo eine Ressource in der Hierarchie steht, hängt vom Nutzungskontext ab — nicht von der Datenbankstruktur.

# HTTP-Semantik für APIs

---

**Leitfrage:** Wie helfen Methoden, Statuscodes und Header dabei, API-Verhalten ohne Zusatzprotokolle klar und maschinenlesbar zu machen?

# Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

**Aufgabe:** Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

| Problem                  | Warum                                          | Besser                    |
|--------------------------|------------------------------------------------|---------------------------|
| 200 OK statt 201 Created | 200 sagt „gelesen“, nicht „erzeugt“            | 201 Created               |
| Kein Location-Header     | Client weiß nicht, wo die neue Ressource liegt | Location: /api/orders/789 |
| Kein Cache-Control       | Bestellungen dürfen nicht gecacht werden       | Cache-Control: no-store   |

## Korrigierte Version



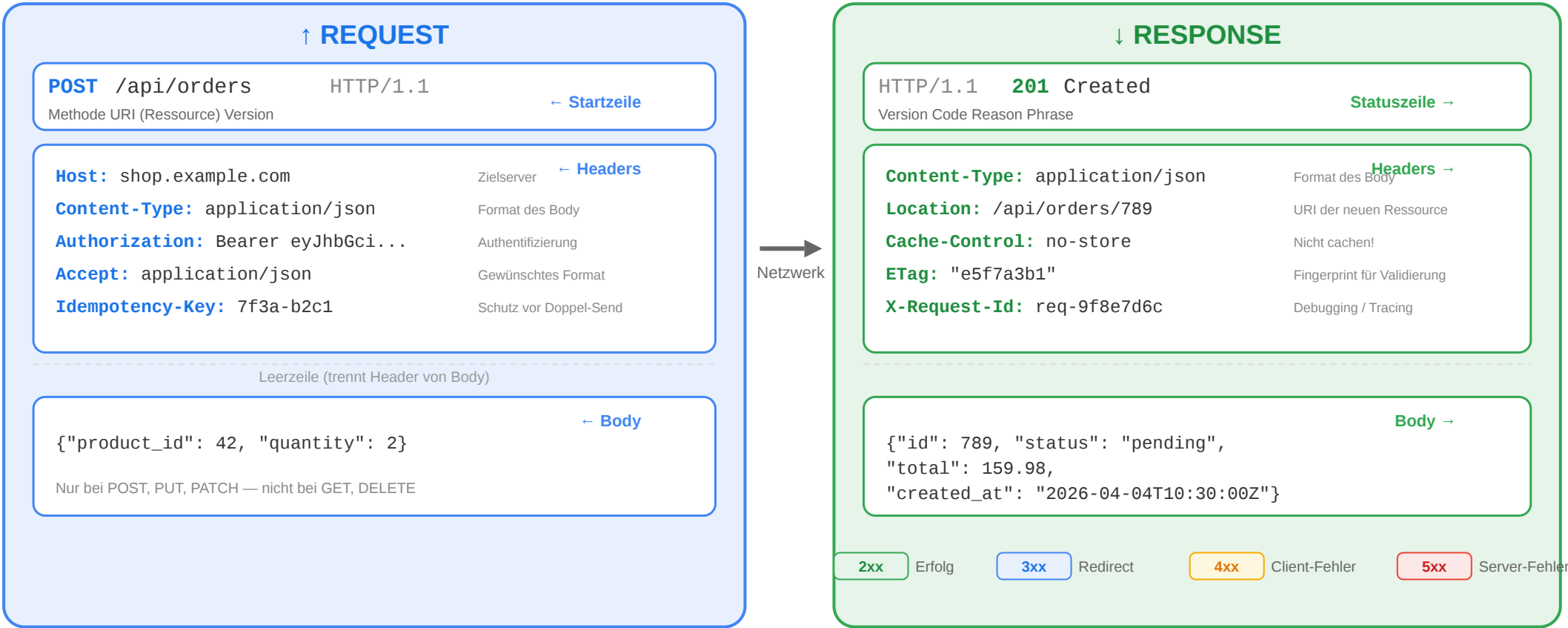
```
HTTP/1.1 201 Created
Location: /api/orders/789
Content-Type: application/json
Cache-Control: no-store
```

```
{"id": 789, "status": "pending", "total": 159.98}
```



# Der vollständige Request-Response-Zyklus

## Anatomie: HTTP Request und Response



# Statuscodes richtig einsetzen

| Situation                              | Richtiger Code         | Häufiger Fehler               |
|----------------------------------------|------------------------|-------------------------------|
| Ressource erfolgreich gelesen          | 200 OK                 | —                             |
| Ressource erfolgreich angelegt         | 201 Created + Location | 200 ohne Location             |
| Erfolgreich, aber kein Body            | 204 No Content         | 200 mit leerem Body           |
| Ungültige Eingabedaten                 | 400 Bad Request        | 500 (ist kein Server-Fehler!) |
| Nicht authentifiziert                  | 401 Unauthorized       | 403                           |
| Authentifiziert, aber nicht berechtigt | 403 Forbidden          | 401                           |
| Ressource nicht gefunden               | 404 Not Found          | 200 mit Fehlermeldung im Body |

## Faustregel:

Der Statuscode sagt dem Client, **was passiert ist**. Der Body erklärt **warum** und **was zu tun ist**.



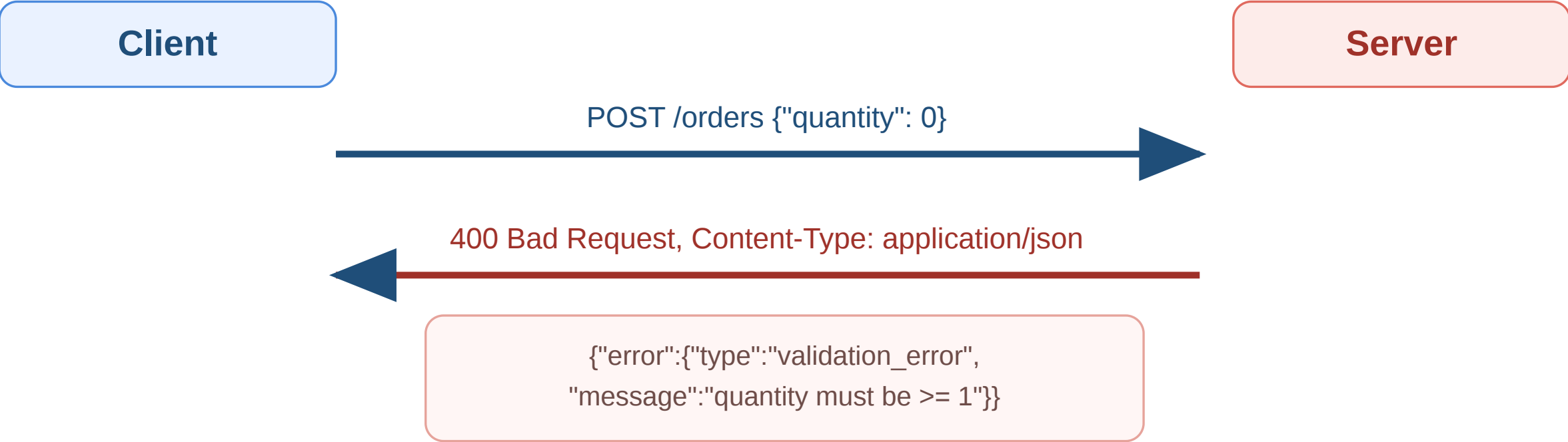
| Situation                          | Richtiger Code | Häufiger Fehler |
|------------------------------------|----------------|-----------------|
| Konflikt (z.B. doppelter Username) | 409 Conflict   | 400             |

# Fehlermodelle: Konsistenz ist alles

```
{
  "error": {
    "type": "validation_error",
    "message": "Die Eingabedaten sind ungültig.",
    "details": [
      { "field": "email", "code": "invalid_format",
        "message": "Keine gültige E-Mail-Adresse" },
      { "field": "quantity", "code": "out_of_range",
        "message": "Muss zwischen 1 und 100 liegen" }
    ]
  }
}
```

| Prinzip              | Umsetzung                                   |
|----------------------|---------------------------------------------|
| Konsistente Struktur | Immer dasselbe JSON-Format für Fehler       |
| Maschinenlesbar      | type und code für automatische Verarbeitung |
| Menschenlesbar       | message für Entwickler und Endnutzer        |
| Spezifisch           | details pro Feld bei Validierungsfehlern    |
| Kein Stacktrace      | Interne Details nie an den Client           |

# Workflow: Fehler sauber kommunizieren



| Situation         | Typischer Code                                 |
|-------------------|------------------------------------------------|
| Ungültige Eingabe | 400 Bad Request oder 422 Unprocessable Content |
| Nicht gefunden    | 404 Not Found                                  |
| Konflikt          | 409 Conflict                                   |



| Situation       | Typischer Code            |
|-----------------|---------------------------|
| Interner Fehler | 500 Internal Server Error |

# Authentifizierung: Wer darf was?

| Mechanismus        | Funktionsweise                                     | Typischer Einsatz                 |
|--------------------|----------------------------------------------------|-----------------------------------|
| API Key            | Statischer Schlüssel im Header                     | Einfache M2M-Kommunikation        |
| Bearer Token (JWT) | Signierter Token im Authorization-Header           | SPAs, Mobile Apps                 |
| OAuth 2.0          | Delegierte Autorisierung über Authorization Server | „Login mit Google“                |
| Session Cookie     | Cookie mit Session-ID                              | Klassische server-gerenderte Apps |

## JWT (JSON Web Token)

Header.Payload.Signature  
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo0Mn0.abc123signature

| Teil      | Inhalt                                            |
|-----------|---------------------------------------------------|
| Header    | Algorithmus und Token-Typ                         |
| Payload   | Claims: user_id, Rollen, Ablaufzeit               |
| Signature | Kryptografische Signatur → Manipulation erkennbar |

**Diskussionsfrage:** JWT ist stateless — der Server muss keine Session speichern. Aber: Wie widerruft man einen JWT, bevor er abläuft?

# Workflow: Authentifizierung mit 401 und 403



| Status           | Bedeutung                                     |
|------------------|-----------------------------------------------|
| 401 Unauthorized | Keine gueltigen Anmeldedaten vorhanden        |
| 403 Forbidden    | Identität bekannt, aber Zugriff nicht erlaubt |



# Rate Limiting und Idempotenz



# Idempotenz

| Methode | Idempotent? | Warum                                     |
|---------|-------------|-------------------------------------------|
| GET     | ja          | Liest nur                                 |
| PUT     | ja          | Ergebnis ist immer derselbe Zustand       |
| DELETE  | ja          | Zweites Löschen hat keinen Effekt         |
| POST    | nein        | Jeder Aufruf kann neue Ressource erzeugen |

**Szenario:** Ein Nutzer klickt "Jetzt bestellen", die Verbindung bricht ab. Der Client weiss nicht, ob der Server die Bestellung schon verarbeitet hat.

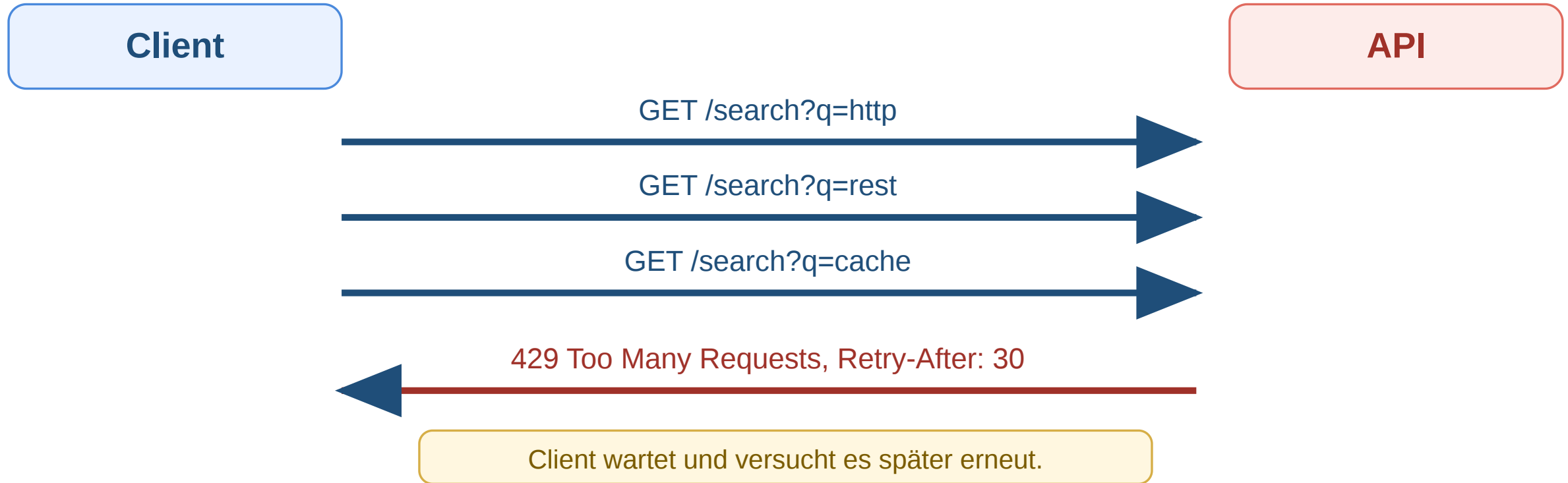
**Loesung:** Idempotency-Key - der Server erkennt den Retry und liefert das Ergebnis des ersten Requests zurueck.

# Rate Limiting

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
```

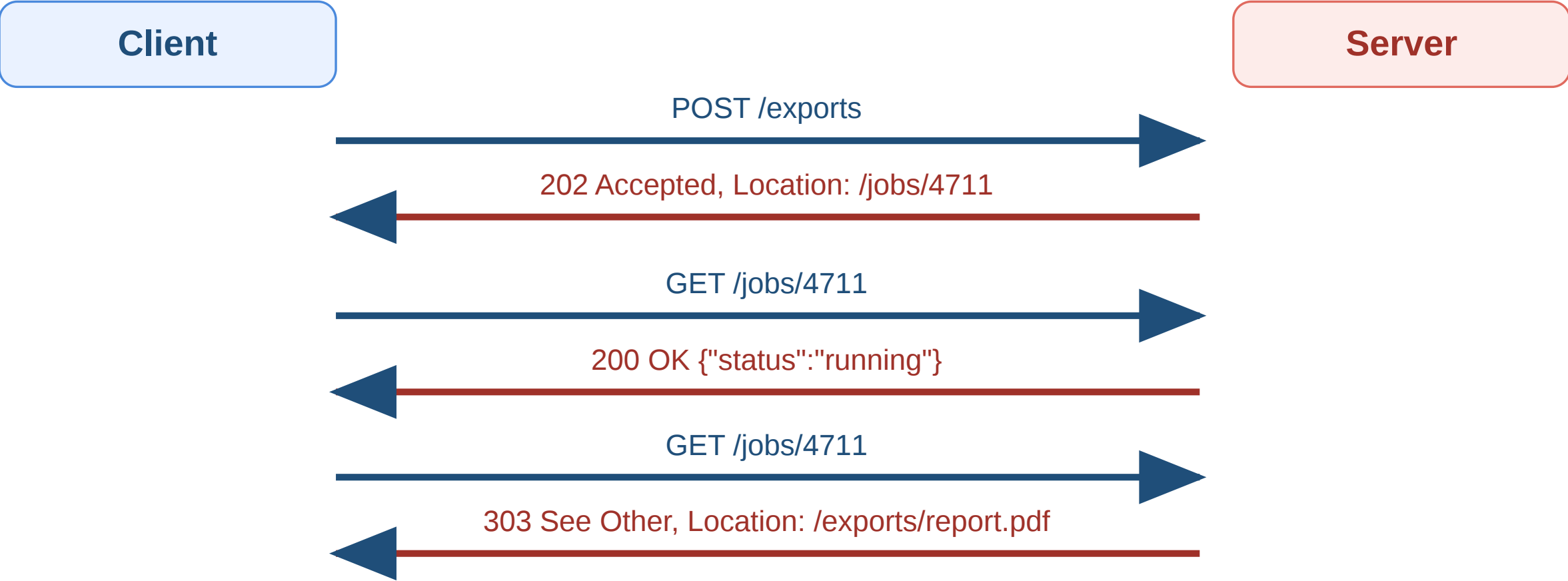
APIs begrenzen Requests pro Zeitfenster. Die Header sagen dem Client **wann** er es erneut versuchen darf.

# Workflow: Rate Limit und Retry-After



| Header / Code         | Bedeutung                                     |
|-----------------------|-----------------------------------------------|
| 429 Too Many Requests | Limit fuer dieses Zeitfenster erreicht        |
| Retry-After: 30       | Neuer Versuch erst nach 30 Sekunden           |
| X-RateLimit-*         | Optionale Zusatzinfos zu Limit und Restbudget |

# Workflow: Asynchrone Verarbeitung mit 202 Accepted



| Phase         | Bedeutung                                         |
|---------------|---------------------------------------------------|
| 202 Accepted  | Auftrag angenommen, aber noch nicht abgeschlossen |
| Job-Ressource | Fortschritt oder Status abfragbar                 |



| Phase         | Bedeutung                                  |
|---------------|--------------------------------------------|
| 303 See Other | Ergebnis liegt nun unter eigener Ressource |

# Evolution, Fehler und API-Qualität

---

**Leitfrage:** Wie entwickelt man APIs weiter, ohne bestehende Clients bei jeder Änderung zu brechen — und wie misst man die Qualität einer API?

# Welche dieser Änderungen brechen Clients?

Der Online-Shop plant ein API-Update. Bewerten Sie jede Änderung:

| Geplante Änderung                                 | Kompatibel? |
|---------------------------------------------------|-------------|
| Neues Feld "created_at" in Produkt-Response       |             |
| Feld "discount" aus Produkt-Response entfernen    |             |
| Neuer optionaler Query-Parameter ?include=reviews |             |
| Feld "userName" umbenennen in "user_name"         |             |
| Neuer Endpunkt GET /products/{id}/recommendations |             |
| Pflichtfeld "phone" in Registrierung hinzufügen   |             |

| Änderung                    | Kompatibel? | Begründung                                      |
|-----------------------------|-------------|-------------------------------------------------|
| Neues Feld in Response      | ja          | Clients ignorieren unbekannte Felder            |
| Feld aus Response entfernen | nein        | Client erwartet "discount", Feld fehlt → Fehler |
| Neuer optionaler Parameter  | ja          | Bestehende Requests funktionieren weiter        |
| Feld umbenennen             | nein        | Client liest "userName", findet es nicht        |



| Änderung               | Kompatibel? | Begründung                                    |
|------------------------|-------------|-----------------------------------------------|
| Neuer Endpunkt         | ja          | Bestehende Endpunkte unverändert              |
| Pflichtfeld hinzufügen | nein        | Bestehende Clients senden "phone" nicht → 400 |

## Prinzip: Additive Evolution

Neue Felder, Endpunkte, optionale Parameter → kompatibel. Entfernen, Umbenennen, Required machen → Breaking Change.



# API-Lebenszyklus

| Phase     | Herausforderung                                           |
|-----------|-----------------------------------------------------------|
| Design    | Ressourcen richtig schneiden, ohne zu viel vorwegzunehmen |
| Implement | Server + Client SDK konsistent halten                     |
| Test      | Nicht nur „funktioniert es?“, sondern „hält der Vertrag?“ |
| Release   | Dokumentation und Versionierung synchron halten           |
| Operate   | Fehler erkennen, bevor Clients sie melden                 |
| Evolve    | Neue Anforderungen einbauen, ohne Bestehendes zu brechen  |

## Robustheitsprinzip (Postel's Law)

*„Be liberal in what you accept, conservative in what you send.“*

**Diskussionsfrage:** Wenn der Server „liberal“ ungültige Eingaben akzeptiert, entsteht ein impliziter Vertrag. Ist das wirklich eine gute Idee?

# Deprecation-Strategie

Wenn Breaking Changes unvermeidbar sind, müssen sie **angekündigt** und **schrittweise** eingeführt werden:

```
Deprecation: true  
Sunset: Sat, 01 Nov 2026 00:00:00 GMT  
Link: </docs/migration-v3>; rel="deprecation"
```

| Header            | Funktion                            |
|-------------------|-------------------------------------|
| Deprecation: true | Endpunkt als veraltet markiert      |
| Sunset            | Ab diesem Datum wird entfernt       |
| Link              | Verweis auf Migrationsdokumentation |

# API-Dokumentation mit OpenAPI

```
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
paths:
  /products/{id}:
    get:
      summary: Produkt abrufen
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Produkt gefunden
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
        '404':
          description: Produkt nicht gefunden
```

| Vorteil                | Erklärung                                     |
|------------------------|-----------------------------------------------|
| Maschinenlesbar        | Automatische Code-Generierung                 |
| Interaktive Docs       | Swagger UI — Entwickler können direkt testen  |
| Contract Testing       | Schema als Grundlage für automatisierte Tests |
| Single Source of Truth | Dokumentation und Implementierung synchron    |



# Observability: APIs im Betrieb

## Die vier goldenen Signale

| Signal     | Frage                            | Online-Shop-Beispiel                     |
|------------|----------------------------------|------------------------------------------|
| Latency    | Wie lange dauern Requests?       | p95 für GET /products < 200ms            |
| Traffic    | Wie viel Last hat das System?    | 5.000 Requests/s zur Cyber-Monday-Spitze |
| Errors     | Wie hoch ist die Fehlerrate?     | 0,1% 5xx ist akzeptabel                  |
| Saturation | Wie nah an der Kapazitätsgrenze? | DB-Connection-Pool bei 85%               |

| Säule    | Was wird gemessen?                          | Werkzeuge             |
|----------|---------------------------------------------|-----------------------|
| Logging  | Einzelne Requests: Wer, Was, Wann, Ergebnis | ELK Stack, Loki       |
| Metriken | Aggregiert: Requests/s, Latenz, Fehlerrate  | Prometheus, Grafana   |
| Tracing  | Ein Request über alle Services hinweg       | Jaeger, OpenTelemetry |



# Bewerten Sie diese API

**Aufgabe:** Prüfen Sie die API des Online-Shops gegen diese Checkliste. Was fehlt?

```
GET /products
  → 200, kein Cache-Control
POST /products
  → 200 statt 201, kein Location
GET /products/999
  → 200 mit {"error": "not found"}
POST /orders
  → 500 bei ungültiger Eingabe
DELETE /products/42
  → 200 mit Body
Fehler-Format: mal {"error": "..."}, mal {"message": "..."}

```

| Problem                     | Verstoß                         | Korrektur                      |
|-----------------------------|---------------------------------|--------------------------------|
| Kein Cache-Control          | Caching unkontrolliert          | Cache-Control: no-cache + ETag |
| 200 statt 201 bei POST      | Falscher Statuscode             | 201 Created + Location         |
| 200 bei nicht gefunden      | Statuscode lügt                 | 404 Not Found                  |
| 500 bei ungültiger Eingabe  | Server-Fehler ≠ Client-Fehler   | 400 oder 422                   |
| Body bei DELETE             | Unnötig                         | 204 No Content                 |
| Inkonsistentes Fehlerformat | Client kann Fehler nicht parsen | Einheitliches Error-Schema     |



API-Design endet nicht beim ersten Release. Die eigentliche Qualität zeigt sich in **Evolution** (additive Changes, Deprecation-Strategie, Postel's Law), **Fehlerkultur** (konsistente Fehlermodelle, spezifische Statuscodes) und **Betriebsfähigkeit** (Observability mit den vier goldenen Signalen). Eine gute API ist vorhersagbar, dokumentiert, beobachtbar und entwickelt sich weiter, ohne bestehende Clients zu brechen.

# Wrap-Up

- **HTTP** ist das Interaktionsmodell des Webs: Methoden drücken Absichten aus, Statuscodes kommunizieren Ergebnisse, Header steuern Caching, Sessions und Sicherheit.
- **HTTP-Repräsentationen** machen sichtbar, in welchem Format eine Ressource übertragen wird und wie Clients und Server dieses Format aushandeln.
- **REST** nutzt diese HTTP-Mechanismen konsequent für APIs — Ressourcen statt Funktionsaufrufe, einheitliche Schnittstelle statt proprietärer Protokolle.
- **Gutes API-Design** beginnt bei sauber geschnittenen Ressourcen, nutzt HTTP-Semantik vollständig und plant Evolution von Anfang an mit.
- **Betrieb** erfordert Observability, Rate Limiting und konsistente Fehlermodelle.

Die nächste Vorlesung zeigt, wie diese APIs **serverseitig implementiert** werden — am Beispiel des Java-Web-Stacks.